Università degli Studi di Messina
Dipartimento MIFT - Corso di Laurea Triennale in Informatica (L-31)

Laboratorio di Reti e Sistemi Distribuiti

HANDS-ON 15:
Implementazione del Flooding Algorithm su Processi POSIX

Massimo Mantineo
Matricola: 541924

Messina, A.A. 2024/2025

Indice

1 Problem Statement **2**

2 Stato dell'Arte **2**

3 Metodologia ed Implementazione **2**
3.1 Topologia ed Avvio 2
3.2 stf() e Multiplexing 3
3.3 Istruzioni di Esecuzione 3

4 Risultati **3**

5 Conclusioni e Sviluppi Futuri **4**

1 Problem Statement

L'Hands-On 15 richiede la dismissione dei costrutti standard d'Ateneo tramite `Pthreads` o i classici Web Sockets TCP, allo scopo di studiare la comunicazione Inter-Processo locale di Linux (IPC) tramite le **Named Pipes** (conosciute come FIFO). L'applicazione designata è la programmazione distribuita di un algoritmo "Flooding", in cui viaggiano messaggi da un Iniziatore (Agente) all'insegna di vicini collegati direzionalmente su un grafo, finchè tutta la rete non è informata. A un nodo (*Processo isolato generato da una Fork()*) e alle comunicazioni sui suoi archi fisici (*Le FIFO in /tmp/*) dovranno asserragliarsi le funzioni `msg()` ed `stf()`.

2 Stato dell'Arte

A livello concettuale, le IPC Pipelines vengono comunemente introdotte da Ritchie.

- **Anonymous vs Named Pipes**: A differenza di 'pipe(fd)', i 'mkfifo()' esistono nel Filesystem Linux e possono essere ispezionati ed aperti dai binari isolatamente solo per nome.
- **Modello a Nodi / Agenti**: Isolare la computazione usando 'fork()' è imperativo in un lab sui grafi, poiché la memoria separata previene trucchi di lettura "illegale" (come array globali condivisi ai thread). L'unico modo in cui il Nodo 2 può sapere cosa ha pensato il Nodo 0 è aspettare sulla propria Pipeline.
- **Costrutti Formali**: L'algoritmo di Santoro identifica il Message Generator ed il valutatore interno (State Transition Function, `stf`).

3 Metodologia ed Implementazione

3.1 Topologia ed Avvio

Trattandosi di uno scenario aperto, abbiamo optato per formalizzare un tipico **Grafo a Diamante di 4 Nodi**: L'Agent 0 (Iniziatore) è connesso asimmetricamente ad 1, 2. Gli Agent 1 e 2 si rivolgono entrambi al nodo di base 3.

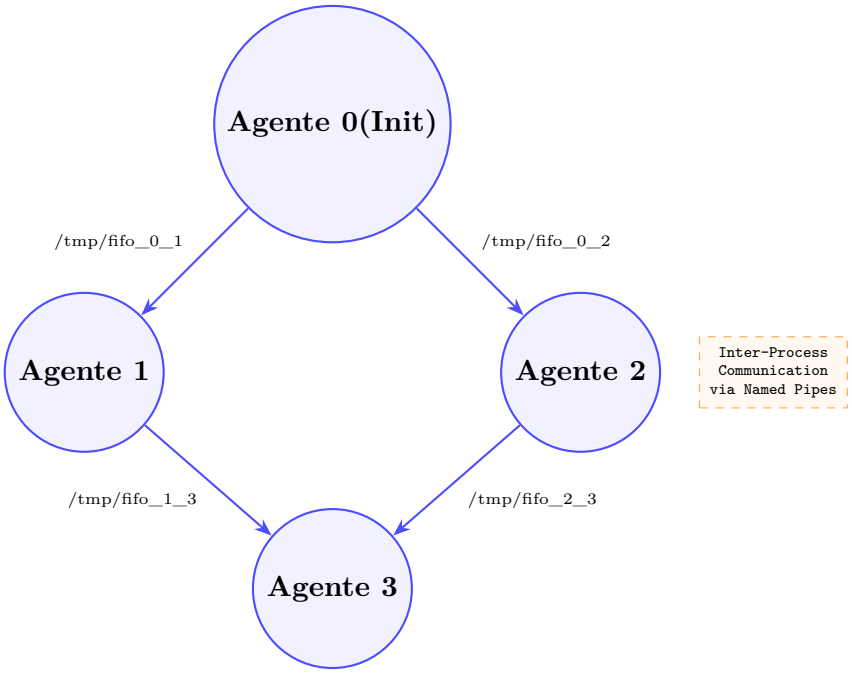

Figure 1: Topologia distribuita a diamante per la simulazione dell'algoritmo di Flooding.

Dal parent `main()` del C, il master lancia ciclicamente la chiamata di Sistema `mkfifo("/tmp/fifo_src_dst", 0666)` generando i tubi direzionali:

Creazione Dinamica

```
for(int i = 0; i < MAX_AGENTS; i++) {
    for(int j = 0; j < num_neighbors[i]; j++) {
        char fname[64];
        sprintf(fname, "/tmp/fifo_%d_%d", i,
           neighbors[i][j]);
        mkfifo(fname, 0666);
    }
}
```

Dopo `fork()` parallela, ogni figlio agisce come Agente e non può più parlare con il fratello se non usando la pipeline.

3.2 stf() e Multiplexing

Le FIFO in ambiente Linux subiscono la natura *Bloccante* della Syscall `open(... , O_RDONLY)`. Quando si chiamano multipli Open, se il vicino non avvia la controparte Write il Processo freezza. È stata utilizzata la bandiera `O_RDWR` (che Linux autorizza per risolvere i deadlock di bootstrap) per poi gettare i rispettivi FD dentro un `fd_set` processato da `select()`.

State Transition Function

```c
void stf(int id, int* state, int sender, const char* info) {
    if (*state == 0) { // PASSIVE TO DONE
        *state = 1;
        for(int i = 0; i < num_neighbors[id]; i++) {
            int neigh = neighbors[id][i];
            if (neigh != sender) msg(id, neigh, info);
        }
    }
}
```

3.3 Istruzioni di Esecuzione

L'applicativo viene gestito interamente tramite Makefile:

Build Run

```
$ make
$ ./programma
```

4 Risultati

Avviando la Routine, la stampa al terminale esalta coerentemente la logica asincrona dei 4 nodi isolati e del Flooding, in cui i Nodi 2 e 3 alla fine scartano i messaggi ritardatari pervenuto dell'altro braccio.

Simulazione Algoritmo di Flooding

```
massimo@PORTATILE-Linux:~/Scrivania/LabReSid24-25/hands-on/ho15$
   ./flooding
--- Avvio Algoritmo di Flooding Centralizzato ---
Topologia: 4 Nodi a Diamante (0 Initiator)

=> [AGENTE 0] INIZIATORE: Genero il messaggio di Flooding
   iniziale.
    -> [AGENTE 0] Propaga messaggio al Nodo 1
    -> [AGENTE 0] Propaga messaggio al Nodo 2
[AGENTE 1] RICEZIONE da Nodo 0: 'ALLERTA_METEO'. TRASIZIONE
   a DONE e Propagazione.
    -> [AGENTE 1] Propaga messaggio al Nodo 3
[AGENTE 2] RICEZIONE da Nodo 0: 'ALLERTA_METEO'. TRASIZIONE
   a DONE e Propagazione.
    -> [AGENTE 2] Propaga messaggio al Nodo 3
[AGENTE 3] RICEZIONE da Nodo 1: 'ALLERTA_METEO'. TRASIZIONE
   a DONE e Propagazione.
    -> [AGENTE 3] Propaga messaggio al Nodo 2
[AGENTE 2] SCARTO msg da Nodo 3 (Gia' elaborato).
[AGENTE 3] SCARTO msg da Nodo 2 (Gia' elaborato).
[AGENTE 2] Spegnimento.
[AGENTE 1] Spegnimento.
[AGENTE 3] Spegnimento.
[AGENTE 0] Spegnimento.
```

A fine esecuzione, il master rimuove ordinatamente dalle cartelle UNIX `/tmp/` la sequela di FIFO in disuso per mantenere il Filesystem esente da clutters.

5 Conclusioni e Sviluppi Futuri

L'algoritmo riproduce con precisione estrema un albero decisionale DFS-like senza attese esterne se non quelle bloccanti native delle pipe POSIX.
Sviluppi futuri ideali:

- **Polimorfismo**: Leggere tramite argomento iniziale il layout di un Grafo qualsiasi esportato dal Software NodeRED o passati via struct JSON per dinamicizzare l'orchestratore, anziché codificare la topologia esplicitamente nel C.
- **Messaggistica Bidirezionale pura**: Usare protocolli a coda per stabilire il Teardown anziché usare un *Timeout empirico* (Il nodo termina dopo N cicli morti dalla `select`). Si eviterebbero cicli clock rubati a vuoto dallo scheduler OS.